

Relatório de Desenvolvimento

VettiFlow 1.0 — Integração Protheus

Emitido em 31 de julho de 2026 | Branch **integracao-protheus** | Último commit caff6dd (03/07/2026)

Resumo executivo

Esta branch implementa a primeira integração real entre o VettiFlow e o Protheus. O app deixou de ser um protótipo isolado: agora lê dados reais do ERP (15.318 OPs, 2.812 empenhos, 707 produtos), permite solicitar abertura de OP, alteração de empenhos e transferência entre armazéns, e envia essas mutações para uma API FastAPI que grava nas tabelas do Protheus. O fluxo opera em duas fases: as mutações são **armazenadas** na API sem tocar o ERP, e só são **aplicadas** quando a Responsável finaliza a OP — garantindo que nada é escrito no Protheus sem confirmação humana. A filial de operação, antes fixa em 04, agora é selecionável na barra superior.

1. Situação geral

Indicador	Estado
Código Dart (app)	75 arquivos — aproximadamente 30.934 linhas
Código Python (API)	6 arquivos — 982 linhas (app/) + 551 linhas (tests/)
Análise estática (flutter analyze)	Sem problemas encontrados
Testes Flutter (flutter test)	64 de 64 aprovados
Testes API (pytest)	18 de 18 aprovados
Assets do Protheus	4 JSONs (15.318 OPs, 2.812 empenhos, 707 produtos, 15 armazéns)
Plataformas suportadas	Windows, Web, Android e macOS (base de código única)
Arquitetura	flutter_bloc (Cubit) + Provider + padrão Repository + FastAPI

2. O que foi entregue nesta branch

2.1 Dados reais do Protheus

Os dados foram extraídos do banco vettip12 (cópia do Protheus migrada para PostgreSQL local em 30/07/2026) e empacotados como assets JSON no app. O MockOpRepository (313 linhas de dados fictícios) foi removido e substituído por repositórios que leem esses assets.

Asset	Registros	Origem no Protheus
ordens.json	15.318 OPs	SC2 (ordens de produção)
empenhos.json	2.812 empenhos	SD4 (empenhos das OPs em aberto)
produtos.json	707 produtos	SB1 (cadastro de produtos)
armazens.json	15 armazéns	NNR (armazéns com saldo)

Novos repositórios criados: ProtheusOrderRepository (96 linhas), EmpenhoRepository (60), ProductCatalogRepository (122), WarehouseRepository (51).

2.2 Modelos de domínio do Protheus

Quatro novos modelos foram criados para representar os dados do ERP:

Modelo	Arquivo	Linhas
ProtheusOrder	data/models/protheus_order.dart	61
ProtheusEmpenho	data/models/protheus_empenho.dart	76
Warehouse	data/models/warehouse.dart	78
PendingMutation	data/models/pending_mutation.dart	570

O PendingMutation é o modelo mais complexo: representa uma mutação pendente (abertura de OP, alteração de empenho ou transferência) com ciclo de vida **pendente** → **enviando** → **armazenado** → **aplicado**.

2.3 Fila de mutações e projeção

O PendingMutationStore (294 linhas) é a peça central da integração no lado do app. Ele:

- Armazena mutações em cache local (localStorage) para operação offline
- Projeta o estado efetivo (empenhosEfetivos, saldosEfetivos) por cima do retrato do ERP — o operador vê o resultado das suas alterações antes de elas chegarem ao Protheus
- Separa mutações por status: pending (aguardando envio), stored (na API), sent (aplicadas)

2.4 Telas de operação Protheus

Tela / Componente	Arquivo	Linhas
Fila do Protheus	ui/protheus/fila_protheus_page.dart	508
Transferência entre armazéns	ui/protheus/transferencia_dialog.dart	488
Editor de empenhos	ui/shared/widgets/empenho_editor.dart	490
Busca de produto	ui/shared/widgets/produto_busca.dart	414
Empenhos da OP	ui/protheus/empenhos_op_dialog.dart	376
Seletor de OP do Protheus	ui/shared/widgets/protheus_op_picker.dart	291
Solicitação de nova OP	ui/shared/widgets/solicitacao_op_form.dart	250

2.5 Fluxo em duas fases (API v0.2.0)

A integração segue um modelo de staging: nada é escrito nas tabelas do Protheus no momento em que o operador faz o pedido. O fluxo é:

Fase	Endpoint	O que acontece
1. Armazenar	POST /api/v1/mutations	Mutações são gravadas na tabela <code>vf_mutations</code> com status armazenado . Nenhuma tabela do Protheus é tocada.
2. Finalizar	POST /api/v1/finalizar	A Responsável confirma. Cada mutação é aplicada nas tabelas (SC2/SD4/SB2) em sua própria transação. Status vira aplicado .

Cada mutação tem um id único que serve de chave de idempotência: reenviar para /mutations devolve o status atual em vez de duplicar.

2.6 API do Protheus (repositório separado)

Repositório: ~/StudioProjects/vetti_flow_api. Python + FastAPI + psycopg3, rodando contra o banco vettip12 (PostgreSQL local).

Método	Rota	Descrição
GET	/api/v1/health	Status e onde a API grava
POST	/api/v1/mutations	Armazena lote (não aplica ainda)
POST	/api/v1/finalizar	Aplica mutações armazenadas no Protheus
GET	/api/v1/mutations/{id}	Status de uma mutação
GET	/api/v1/ops/{op}/empenhos	Empenhos da OP (direto da SD4)

Método	Rota	Descrição
GET	/api/v1/ops/{op}/armazenadas	Mutações armazenadas para uma OP
GET	/api/v1/produtos/{cod}/saldos	Saldo por almoxarifado (direto da SB2)

Arquivo	Linhas	Responsabilidade
app/main.py	365	Endpoints, orquestração de armazenar e finalizar
app/protheus.py	335	Lógica de escrita em SC2, SD4 e SB2
app/schemas.py	126	Modelos Pydantic (mutações, resultados)
app/db.py	128	Pool de conexões psycopg3
tests/test_mutations.py	551	18 testes contra banco real, com limpeza

2.7 Seleção de filial

A constante `filialOperacao = '04'` foi removida. No lugar, o `FilialStore` (39 linhas) persiste a filial selecionada em `localStorage`. Um dropdown “Filial 03” / “Filial 04” aparece na barra superior de todas as telas de operação (firmware, teste, expedição).

2.8 Alterações em arquivos existentes

Além dos 31 arquivos novos, 21 arquivos existentes foram modificados (+1.547 linhas, -991 linhas). As alterações mais significativas:

Arquivo	+/-	O que mudou
production_flow_store.dart	-356	Simplificado: delega persistência para <code>LocalJsonPersistence</code>
nova_op_dialog.dart	+336	Duas abas: adotar OP existente ou solicitar nova OP
warehouse_page.dart	+308	Integrado com empenhos reais e transferência entre armazéns
widget_test.dart	+309	64 testes (antes 22) cobrindo novos fluxos
ordens_view.dart	+242	Kanban agora mostra OPs reais do Protheus
vetti_top_bar.dart	+106	Seletor de filial e botão da fila do Protheus
dashboard_page.dart	+90	Integração com <code>ProtheusOrderRepository</code> e aba Armazenadas

Arquivo	+/-	O que mudou
vetti_flow_app.dart	+68	Providers para FilialStore, PendingMutationStore, MutationSyncService
mock_op_repository.dart	-313	Removido — substituído por dados reais

3. O que cada mutação faz nas tabelas do Protheus

Tipo	Tabelas tocadas	Descrição
aberturaOp	SC2 + SD4 + SB2	Cria a OP, insere empenhos e reserva saldo (b2_qemp). Sem empenhos no pedido, explode a estrutura do produto (SG1).
empenho	SD4 + SB2	Adiciona ou altera empenhos da OP e atualiza reserva de saldo.
transferencia	SB2	Move saldo (b2_qatu) entre dois almoxarifados.

4. Armadilhas das tabelas do Protheus

Lições aprendidas durante a implementação que valem para qualquer escrita futura:

- R_E_C_N_O_ não é sequence:** Chave primária gerenciada pelo ERP por fora. Inserir sem calcular max+1 deixa tudo em zero e colide.
- Todas as colunas NOT NULL sem default:** A SC2 tem 151 colunas. INSERT montado via information_schema.
- Datas são varchar(8) YYYYMMDD:** Não são campos date — são strings.
- Exclusão lógica (d_e_l_e_t_ = '*'):** Esquecer o filtro traz registros mortos.
- Vigência na SG1 (G1_INI/G1_FIM):** Sem filtro vêm componentes de revisões antigas.
- Saldo negativo é permitido:** Material a caminho. A API não pode ser mais rígida que o ERP.

5. Cobertura de testes

5.1 Flutter (64 testes)

Os testes cobrem navegação, renderização por viewport, regras do fluxo de produção, e os novos cenários de integração: projeção de empenhos efetivos, serialização da fila de mutações, persistência cross-platform via LocalJsonPersistence.

5.2 API (18 testes)

Os testes rodam contra o banco vetti12 real e desfazem o que fizeram. Os 4 novos testes do fluxo em duas fases verificam:

- `test_mutations_armazena_sem_aplicar`: POST /mutations retorna **armazenado**, nenhuma OP criada

- `test_finalizar_aplica_no_protheus`: armazenar → finalizar → retorna **aplicado** com `protheusRef`
- `test_reenviar_armazenada_retorna_mesmo_status`: idempotência do armazenamento
- `test_finalizar_mutacao_inexistente_retorna_erro`: tratamento de erro

6. Gargalos resolvidos desde o relatório anterior

O relatório de 23/07/2026 listava 5 gargalos que travavam o piloto. Esta branch resolve diretamente os itens #1 e #4:

- **#1 Persistência apenas na Web** — LocalJsonPersistence funciona em todas as plataformas. **Resolvido.**
- **#4 Integração Protheus não iniciada** — Dados reais, fila de mutações e API funcional. **Resolvido.**
- **#2 Estado não compartilhado** — API centraliza mutações; falta o fluxo de produção. **Em andamento.**
- **#3 Senhas em texto plano** — Sem mudança nesta branch. **Pendente.**
- **#5 Telas grandes demais** — Sem refatoração estrutural nesta branch. **Pendente.**

7. Arquitetura da integração

O diagrama abaixo mostra como as peças se conectam:

Cama da	Componente	Responsabilidade
UI	Fila do Protheus, EmpenhoEditor, TransferenciaDialog, NovaOpDialog	Operações do operador: solicitar OP, editar empenhos, transferir material, visualizar fila
Store	PendingMutationStore FilialStore	Cache local das mutações + projeção de estado efetivo. Filial selecionada persistida.
Sync	MutationSyncService ProtheusSyncClient	Empurra fila para a API (sync) e solicita finalização. Cliente HTTP com idempotência.
API	FastAPI (main.py)	Fase 1: armazena em <code>vf_mutations</code> . Fase 2: aplica via <code>protheus.py</code> nas tabelas do ERP.
ERP	<code>protheus.py</code> → SC2/SD4/SB2	Escrita real nas tabelas. Quando o Protheus de produção entrar, só este arquivo muda (chama MATA650 e afins).

8. Números da branch

Métrica	Valor
Arquivos novos (app)	31
Arquivos modificados	21
Arquivos removidos	4 (MockOpRepository + persistência antiga)

Métrica	Valor
Linhas adicionadas (app)	+5.552 (novos) + 1.547 (modificados)
Linhas removidas (app)	-991 (modificados) - 347 (removidos)
Saldo líquido (app)	+5.761 linhas
Código da API	982 linhas (app/) + 551 linhas (tests/)
Total de testes	64 Flutter + 18 API = 82 testes

9. Próximos passos recomendados

Fase 1 — Piloto em produção

- **Compartilhar OPs entre máquinas** (Crítica) — Hoje cada estação guarda seu próprio fluxo. A API já centraliza mutações; falta sincronizar o estado do fluxo de produção.
- **Apontar a API para o Protheus real** (Crítica) — Trocar protheus.py para chamar as rotinas do ERP (MATA650 etc.) em vez de escrever diretamente nas tabelas. O contrato com o app não muda.
- **Commit e merge desta branch** (Alta) — Todo o código está verificado e testado mas não foi commitado ainda.

Fase 2 — Robustez

- **Remover credenciais do código** (Alta) — Usuários e PINs estão literais no código-fonte. Migrar para autenticação via endpoint.
- **Tratamento de conflitos** (Média) — Quando dois operadores alteram a mesma OP, quem ganha? Hoje a última mutação finalizada prevalece.
- **Monitoramento da fila** (Média) — Alertar quando mutações ficam represadas por muito tempo sem finalizar.

Conclusão

O VettiFlow deixou de ser um protótipo de interface e passou a ser uma ferramenta conectada ao ERP. O operador do chão de fábrica agora solicita abertura de OP, ajusta empenhos e transfere material sem telefonar para quem tem acesso ao Protheus. A Responsável confirma antes de qualquer escrita real no ERP. Com 82 testes passando e o fluxo validado contra o banco vettip12, o próximo passo natural é apontar a API para o Protheus de produção e compartilhar o estado entre as estações.